Learning Branch-and-Bound Strategies for Solving Mixed-Integer Programs in the Context of Robot Motion Planning

Abhishek Cauligi [*1] **Preston Culbertson** [*2] **Joseph Lorenzetti** [*1]

Abstract

Mixed-integer linear programs (MILPs) are an expressive class of optimization problems which can model important robotics problems such as collision avoidance and obstacle navigation. However, MILPs are NP-complete in the general case, and often not tractable in real-time, which limits their practical application.

In this work, we propose an inverse reinforcement learning (IRL) framework which aims to learn good node selection heuristics for branch-and-bound, a class of algorithms used to solve MILPs. We propose training a neural-network-based policy to score open nodes in a branch-and-bound tree, which will allow the solver to quickly select nodes corresponding to a correct solution. We train this policy using expert demonstrations from optimal solutions computed offline. We study our policy's performance empirically on a class of robotic navigation problems, and show a speedup both in LPs solved, and solution time, over a common baseline heuristic.

1. Introduction

Discrete and combinatorial optimization problems have been used to formulate and solve challenging problems within a broad range of applications including robotics (Landry et al., 2016; Richards et al., 2002; Schouwenaars et al., 2001), autonomous mobility-on-demand (Tsao et al., 2019; Iglesias et al., 2018), chemical process scheduling (Floudas & Lin, 2005; Pinto & Grossmann, 1995), power network design (Mashayekh et al., 2017; Amini et al., 2015), production planning (Pochet & Wolsey, 2006), and more.

Specifically, these problems are often posed as mixed-integer programs, which are optimization problems with convex objective functions, convex constraints, and where a subset of the decision variables are constrained to be integral (a non-convex constraint). One instance of this problem type is the widely used mixed-integer linear program (MILP), which has a linear cost function and linear constraints. For a review on integer programming see (Conforti et al., 2014). While the inclusion of integrality constraints within linear programs make MILPs extremely expressive, it also makes them NP-complete and exceptionally difficult to solve in the worst case.

Fortunately there exist strategies for solving MILPs in reasonable time, most of which are based on the branch-and-bound algorithm. The branch-and-bound algorithm involves sequentially solving convex relaxations of the MILP to efficiently search for the global optimum (for an overview see Section 2). Crucially, the algorithm's performance is highly dependent on the sequencing of the convex relaxations it solves, which has led to extensive research and engineering for designing good heuristics to guide the algorithm. These efforts have been very successful and have led to exceptional performance by solvers such as CPLEX, Gurobi, CBC, and more.

While the heuristics employed by these solvers have proven to be very useful, they are generalized and therefore potentially agnostic to useful problem-specific information. The goal of this work is therefore to develop a framework for *learning* heuristics that can leverage problem-specific information. Such a framework would be useful in settings (such as those mentioned previously) where a specific MILP is repeatedly solved with only slight variations, and where the underlying problem has some structure that could be exploited. We model the branch-and-bound algorithm as an agent in a sequential decision making problem, where the agent chooses which convex relaxations to solve at each step. The agent's policy is developed using techniques from reinforcement learning, and the learned policy yields the branch-and-bound heuristic. For simplicity we restrict our consideration to the problem of robotic motion planning, where the integers are binary valued. See Section 2 for the MILP formulation of the motion planning problem we consider.

[1]Department of Aeronautics and Astronautics, Stanford University, Stanford, CA, USA [2]Department of Mechanical Engineering, Stanford University, Stanford, CA, USA.

2. Background

2.1. Binary-Integer Linear Program

The motion planning problem considered in this work can be represented as a binary-integer linear program. This type of optimization problem has a linear objective, linear constraints, and a subset of the decision variables constrained to be binary integers. The general form of this problem is:

$$\begin{aligned} J^* = \underset{x,y}{\text{minimize}} \;\; & c^T[x,y]^T \\ \text{subject to} \;\; & A[x,y]^T \leq b, \\ & y_i \in \{0,1\}, \;\; i \in [1,\dots,n_y] \end{aligned} \tag{1}$$

where $x \in \mathbb{R}^{n_x}$ and $y \in \mathbb{R}^{n_y}$ are the decision variables and the elements of y are binary integers. The vector $c \in \mathbb{R}^{n_x+n_y}$ defines the cost function and the matrix $A \in \mathbb{R}^{n_c \times (n_x+n_y)}$ and vector $b \in \mathbb{R}^{n_c}$ define the problem constraints.

2.2. Branch-and-bound

The standard method for solving problems of the form (1) utilizes the branch-and-bound algorithm, which operates by sequentially building an enumeration tree of relaxed problems to the original problem. The root node of this tree is defined as the linear program resulting from relaxing all integrality constraints. For problems containing only binary integers, subsequent nodes are linear programs with some subset of the integer variables fixed to specific values (either 0 or 1) and the others having only the relaxed constraint $y_i \in [0,1]$. Each unique node in the branch-and-bound tree can then be identified using the set of integers variables that are fixed in the nodes' relaxed linear program. Therefore we use the notation $LP\{y_{i,0}, y_{j,1}\}$ to represent the node whose linear program is generated by taking the root node and appending the constraints $y_i = 0$ and $y_j = 1$. The root node is denoted as $LP\{\emptyset\}$, and an arbitrary node will be denoted as $LP\{\cdot\}$.

As mentioned, nodes are sequentially added to the branch-and-bound tree. The process starts with the root node $LP\{\emptyset\}$, where a branching variable y_i is then chosen. Branching on y_i then creates two new nodes: one with the added constraint $y_i = 0 \rightarrow LP\{y_{i,0}\}$ and a second with the constraint $y_i = 1 \rightarrow LP\{y_{i,1}\}$. The tree continues to grow by repeating this process and choosing new branching variables that have not been previously selected. If the process continued to completion each leaf of the tree would be a node whose partial problem has all y_i fixed, and therefore there would be 2^{n_y} leaf nodes (corresponding to all possible permutations of the integer solution). The critical component that defines the branch-and-bound algorithm is that searching an exponential (2^{n_y}) number of nodes can potentially be avoided by early identification of branches that cannot possibly contain the optimal solution. If such an identification occurs the branch can be *pruned* and no further exploration of that branch is required.

Methods for pruning branches rely on the critical observation that the feasible set of solutions to the linear program for a given node $LP\{\cdot\}$ is a subset of the feasible set for its parent node linear program (since the child LP was generated by only adding constraints). This leads us to the first condition for pruning:

(1) A node $LP\{\cdot\}$ can be pruned from the tree if the linear program is infeasible.

This is apparent because all future nodes on that branch must also be infeasible and therefore no solution to the original MILP (1) exists further along that branch. Additional conditions for pruning arise from bookkeeping the lowest known upper bound $\bar{J}^*$ on the solution to the original MILP (1), J^*, such that $\bar{J}^* \geq J^*$. In practice the lowest known upper bound corresponds to the best feasible solution to the MILP (1) found so far (referred to as the *incumbent* solution $(\bar{x}^*, \bar{y}^*)$) and its value is often initialized to be $\bar{J}^* \leftarrow \infty$ with $(\bar{x}^*, \bar{y}^*) \leftarrow (\emptyset, \emptyset)$. The bookkeeping of the upper bound $\bar{J}^*$ leads to two additional pruning conditions, one of which is:

(2) A node $LP\{\dots\}$ can be pruned from the tree if the linear program solution (x,y) is a feasible solution to the original MILP (1).

This is apparent because the solution must remain feasible (and therefore optimal) for all children linear programs. Further, since the solution (x,y) to $LP\{\cdot\}$ is feasible for MILP (1), the value J of $LP\{\cdot\}$ can be compared to the lowest known upper bound $\bar{J}^*$. If $J < \bar{J}^*$ then a new best solution has been found and we replace the upper bound $\bar{J}^* \leftarrow J$ and incumbent $(\bar{x}^*, \bar{y}^*) \leftarrow (x,y)$. We can now present the last condition for pruning:

(3) A node $LP\{\cdot\}$ can be pruned from the tree if the linear program value J is no better than the lowest known upper bound $\bar{J}^*$, such that $J \geq \bar{J}^*$.

Since the feasible set for all children of the node $LP\{\cdot\}$ is a subset of the feasible set for the node $LP\{\cdot\}$, the value J is a lower bound for all possible nodes further along the branch. Therefore because $J \geq \bar{J}^*$ there cannot exist a solution in the current branch that will replace the incumbent solution.

With these pruning conditions, the branch-and-bound algorithm continues to build the enumeration tree until there are no nodes $LP\{\cdot\}$ left to branch from. At this point the incumbent solution must be the globally optimal solution to the MILP (1), (i.e. $J^* = \bar{J}^*$ and $(x^*, y^*) = (\bar{x}^*, \bar{y}^*)$). This algorithm is outlined in Algorithm 1, where the set $\mathcal{L}$ contains all nodes $LP\{\cdot\}$ that still need to be branched, referred to as *active* nodes.

Algorithm 1 Branch-and-Bound

Input: MILP (1)
$LP\{\emptyset\} \leftarrow$ rootNode(MILP (1))
$\mathcal{L} \leftarrow \{LP\{\emptyset\}\}, \;\; \bar{J}^* \leftarrow \infty, \;\; (\bar{x}^*, \bar{y}^*) \leftarrow (\emptyset, \emptyset)$
repeat
  $LP\{\cdot\} \leftarrow$ popNode($\mathcal{L}, \dots$)
  $y_i \leftarrow$ chooseBranchVar($LP\{\cdot\}, \dots$)
  for b **in** $\{0, 1\}$ **do**
    $LP\{\cdot, y_{i,b}\} \leftarrow$ branch($LP\{\cdot\}$, $y_i = b$)
    $J, \; (x, y) \leftarrow$ solveRelaxedLP($LP\{\cdot, y_{i,b}\}$)
    if isFeasible($LP\{\cdot, y_{i,b}\}$) **and** isInteger(y) **then**
      if $J < \bar{J}^*$ **then**
        $\bar{J}^* \leftarrow J, \;\; (\bar{x}^*, \bar{y}^*) \leftarrow (x, y)$
      end if
    else if isFeasible($LP\{\cdot, y_{i,b}\}$) **and** $J < \bar{J}^*$ **then**
      $\mathcal{L} \leftarrow \mathcal{L} + \{LP\{\cdot, y_{i,b}\}\}$
    end if
  end for
until $\mathcal{L} = \emptyset$
return $\bar{J}^*$, $(\bar{x}^*, \bar{y}^*)$

2.3. Branch-and-bound Strategies

At each step of the branch-and-bound algorithm two decisions need to occur: the first is to choose a node for branching from the set of active nodes $\mathcal{L}$ (called *node selection*), and the second is to choose which variable y_i at that node should be branched on (called *variable selection*). Developing good strategies for *node selection* and *variable selection* is critical for making branch and bound effective.

Generally speaking, algorithm termination time is directly correlated to the number of nodes that are explored in the tree. Good strategies seek to minimize the number of nodes visited by leveraging the pruning conditions mentioned in Section 2.2[1]. For example, one node selection strategy known as *depth-first search* tries to find a feasible solution to the MILP right away so that $\bar{J}^*$ is decreased. Another node selection strategy is the *best-first search*, which always chooses the node in $\mathcal{L}$ with the lowest J value. This strategy optimistically assumes that searching the best relaxed problems will also lead to finding good incumbent solutions (i.e. MILP feasible solutions that lower $\bar{J}^*$). Both of these commonly used strategies are attempting to find good incumbent solutions early, so as to maximally take advantage of pruning condition (3).

For variable selection, one simple strategy is to choose the variable y_i that is the *most fractional* in the current node. More sophisticated strategies include *strong branching* and *pseudocost branching*. Strong branching probes each variable at a given node by solving additional relaxed linear programs, which empirically leads to good results but increases computation requirements. Pseudocost branching is a technique that seeks to imitate strong branching with less computational effort required, but also can require strong intuition and engineering to work well.

[1] Of course there is also a trade-off between computation time for implementing the strategy and the efficiency gained from pruning.

2.4. Motion Planning Problem

As mentioned earlier, in this work our goal is to learn heuristics to guide the branch-and-bound algorithm that can leverage problem-specific information. To demonstrate our approach we have chosen a fundamental problem in robotics: motion planning in the presence of obstacles.

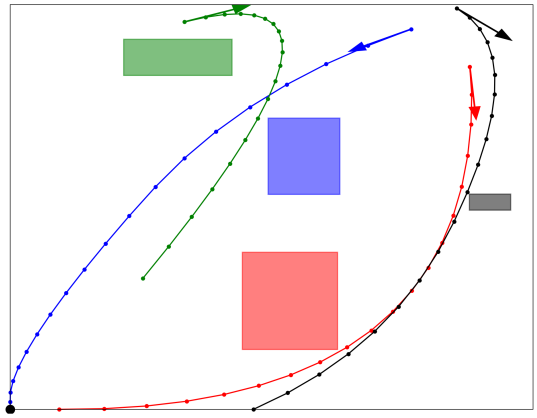

Figure 1. Example obstacle avoidance problems. In each sample trajectory the horizon is $N = 20$ steps, the objective is to go the origin as quickly as possible, and the robot's initial position and initial velocity is shown via the colored arrows. In each case the robot is a double integrator with actuation constraints.

We formulate a problem consisting of a single robot and a single obstacle that the robot must avoid. The robot's task is to plan a trajectory to get from its initial location to a goal location (in this case the coordinate $(0, 0)$). To accomplish this a receding horizon control approach is used, where the robot plans a trajectory over a finite horizon, executes a portion of the determined control, and then replans. This continues in a recursive fashion until the robot has reached the goal location. Receding horizon approaches are very commonly used for trajectory planning, and they enable the use of optimization based techniques to plan optimal paths subject to hard constraints (such as obstacle avoidance). For our robot motion planning problem we use the formulation from (Schouwenaars et al., 2001) to define the optimization problem that is solved at each time

step. For simplicity we are considering a robot that exhibits double integrator dynamics and has actuation constraints. The robot is also constrained to stay within a specific region, and of course must avoid the obstacle. Several sample trajectories resulting from this problem setup and by solving the optimization problem defined in (Schouwenaars et al., 2001) are given in Figure 1.

3. Related Work

Some previous work has been done in the field of learning for discrete optimization. For example (Bengio et al., 2018) surveys learning based methods for combinatorial optimization and (Lodi & Zarpellon, 2017) surveys learning based methods for branching in branch-and-bound algorithms. One specific approach of learning to branch is presented in (Khalil et al., 2016), where they propose a way to learn how to imitate the well known heuristic called "strong branching", which is computationally cumbersome to perform. A machine learning approach to imitate strong branching is also presented in (Alvarez et al., 2017). Another supervised learning approach for augmenting branch-and-bound is given in (Balcan et al., 2018) where they learn how to select the branching procedures for a distribution of problem instances. In all of these methods, the goal is to learn what *variable* to branch on at a specific node, and they do not consider how to design the *node* selection strategies.

Learning good *node* selection strategies is considered in other recent work. For example (Sabharwal et al., 2012) proposes an approach based on Upper Confidence bounds for Trees (UCT). Another approach, based on imitation learning, is given in (He et al., 2014). This approach utilizes an oracle node selection policy (that always chooses to branch on a node that contains the optimal solution) to train two policies. The first policy chooses which node to branch on, and the second chooses whether to prune a node before the standard pruning conditions are met. In this work the chosen features used in the policy only contain information originating from the branch-and-bound algorithm, and do not contain problem specific information. The use of the pruning policy to speed up the search is also sometimes undesirable, because no guarantees can be made on finding the globally optimal solution.

4. Approach

In this work we focus on efficiently guiding the branch-and-bound algorithm by synthesizing a *node selection* policy that is learned from experience. To accomplish this we model the task of selecting nodes as a sequential decision making process and train a policy from expert demonstrations using DAgger (Ross et al., 2011).

Our method takes inspiration from (He et al., 2014), but is different in several ways. Most importantly, (He et al., 2014) is a more general method that does not utilize any problem specific information, whereas our goal is to specifically leverage problem specific information. Our approach also do not consider a pruning policy because we would like to solve the MILP to obtain a certifiably globally optimal solution. Finally, we parameterize our policy as a neural network to make the policy more expressive.

4.1. Branch-and-bound Sequential Decision Process

The branch-and-bound algorithm can be modeled as a sequential decision making process by considering its action space to consist of all possible $(\text{node}, \text{variable})$ pairs that could define the next branching move. The state for this process would generally consist of the entire structure of the tree (all nodes explored thus far). The goal is then to synthesize a policy for selecting the $(\text{node}, \text{variable})$ pair, given the state of the branch-and-bound tree, to maximize some reward. However there are several challenges with representing the algorithm in this way: (1) the state space, consisting of all possible sub-trees that could be enumerated, is massive, (2) the number of possible actions at each step changes as the number of active nodes changes, (3) a reward structure could be challenging to identify.

Inspired by (He et al., 2014), we tackle these challenges by using an imitation learning approach and by simplifying the problem to only consider the *node selection* policy. Imitation learning is well suited to this problem because using traditional reinforcement learning methods in such a massive state space could be extremely difficult. Additionally, by the nature of the algorithm if the optimal solution is already known it is easy to identify a good node selection policy a posteriori, which provides an implicit reward function. We now discuss in further detail the policy that is learned, the expert policy used, and the training method.

4.2. Learned Node Selection Policy

As mentioned, the main challenges faced when developing the node selection policy is that the state space is massive and the number of possible nodes can change at each iteration. In this work we use a value function approximation-like approach to predict a score $S(s, a)$, where s is the state of the branch-and-bound tree and a represents the selected node from all active nodes in $\mathcal{L}$. To address the large state space we assume that the relevant state information can be encoded using a set of features $\phi(s, a)$. This leads to computing the scores using the function approximation $S(s, a) \approx f_\theta(\phi(s, a))$, with parameters θ. The node selection policy at each iteration is then given by

$$\pi(s) = \underset{a \in \mathcal{L}}{\text{argmax}} \; f_\theta(\phi(s, a)) \tag{2}$$

Special care should be taken in selecting the features $\phi(s, a)$ that can accurately represent the state of the branch-and-bound tree. In (He et al., 2014) these features include the lower bound of the current node, node depth, global upper and lower bounds, integrality gap, and a few others. However they do not include features that could capture structure inherent to the problem. In this work, for the robotic motion planning problem, the features include a mix of branch-and-bound features (i.e. node lower bound, node depth, variable assignments) and problem specific features (i.e. robot initial conditions, position/size of the obstacles). In our work the function $f_\theta(\phi(s, a))$ is parameterized as a neural network with two hidden layers of 32 neurons, using ReLU activation.

4.3. Expert Policy

To learn the function $f_\theta(\phi(s, a))$ that can lead to an efficient node selection policy we use an imitation learning approach. This requires an expert policy that should guide the training. Luckily, an expert policy for the branch-and-bound node selection is easy to identify.

Given a MILP, the problem is first solved to global optimality (using any heuristic or commercial solver). Based on the pruning conditions given in Section 2.2, we can see that one way to reduce the number of nodes visited is to immediately build the branch to the globally optimal solution. This would yield the lowest upper bound $\bar{J}^* = J^*$, which provides the best opportunity for pruning other branches visited later using pruning condition (3). Therefore the expert node selection policy, under a given variable selection policy, simply chooses nodes containing the known globally optimal solution in the node's sub-tree. To represent this decision, our expert policy gives a node containing the globally optimal solution in its sub-tree a score of $+1$, and all other nodes a score of -1.

4.4. Policy Training

We use DAgger (Ross et al., 2011) to provide a stable training procedure for the inverse RL task. Given the expert node selection policy, we seek to train the function $f_\theta(\phi(s, a))$ to score nodes equivalently to the expert. At a high level, DAgger interleaves expert and learned actions to allow for exploration under a similar state distribution to that encountered by the learned policy; however all examples are labelled for training per the expert's actions. An outline of our training procedure is provided in Algorithm 2.

During training, we sample a given problem from our dataset, and solve it using a DAgger-like strategy. Note that we interleave expert and learned actions by using an ϵ-greedy style strategy, with ϵ decaying linearly to zero throughout training. We end the episode when the incumbent solution of branch-and-bound is optimal, since node selection has no effect on solve time after finding an optimal solution. The function is trained with an Adam optimizer, using a loss which contains a hinge loss and a regularization term, given by

Algorithm 2 Policy Training with DAgger

```
initialize D, θ
for i in 1 to N do
    sample MILP p, cost J* from dataset
    J̄ ← UB(p)
    while J̄ > J* do
        with probability ε choose a = π*(s)
        otherwise choose a = argmax_ā f_θ(φ(s, ā))
        store (s, π*) in D
        s, J̄ ← bnb(s, a)
        every C steps do
            sample mini-batch M = (s_i, a_i*) from D
            perform gradient descent step on L(M, θ)
    end while
end for
```

$$L(s_i, a_i) = \max\{0, 1 - y_i f_\theta(\phi(s_i, a_i)\} + \lambda||\theta||,$$

where (s_i, a_i) represent a given branch-and-bound state and node, y_i is a label of $+1$ if the node corresponds to the chosen node under the expert policy π^* and -1 otherwise, and λ is a weighting parameter balancing the hinge and regularization terms of the loss. We use $\lambda = 1e - 3$ for all policies.

However, since incorrect nodes (i.e. those not chosen by the expert) are by far more frequently encountered than correct ones, when sampling mini-batches for training, we provide an equal proportion of correct and incorrect nodes to avoid the effects of dataset imbalance. Further, since sequential samples are not i.i.d., we sample training examples randomly from a common experience buffer (which contains examples from all problems solved thus far) to promote generalization across various problems. Finally, problems on which our learned policy performs poorly will generate far more nodes (i.e. take longer to solve) than problems on which our policy does well. This has the virtuous effect of over-representing such problems in the dataset, which provides more opportunities for samples from these problems to be encountered during training.

5. Results

To demonstrate the proposed approach we learn two different *node selection* policies and compare their performance against the expert policy, a simple but commonly used heuristic, and a random policy.

The first policy we learn utilizes the following features: (1) robot initial condition, (2) obstacle position and size, (3)

node variable assignments $y_i = \ldots$, (4) node lower bound J, and (5) node depth. Our hypothesis is that including features (1)-(3) will enable better policies by allowing the network to learn to exploit the problem structure. For example by using the relative position of the robot to the obstacle, or by leveraging the fact that the value of the integer variables tend to be correlated with respect to time. The second policy we learn only contains the branch-and-bound specific features (4) and (5). While we don't expect this to perform as well as our first policy, it could still outperform other heuristics (e.g. as is shown in (He et al., 2014)).

For all of the *node selection* policies presented in these results, the *variable selection* policy was fixed to choosing the most fractional variable. Comparing results using different variable selection heuristics will be an important item of future work. Additionally for both of the learned policies the learned neural network contained 2 layers with 32 neurons each and ReLU activation, followed by a fully connected output layer.

Two measures of performance were used to evaluate success. The first measure of performance is the number of nodes visited, which corresponds to the number of linear program relaxations that were solved. This is an important metric because solving each linear program is generally the most computationally expensive part of the algorithm. The second measure of success was the actual time taken to run the branch-and-bound algorithm to completion. This is an important metric because it is fundamentally the most important criteria for the robotic motion planning problem. While highly correlated with the number of nodes visited, this metric also captures the important fact that using computationally expensive node selection policies can end up eating up the performance margin that is gained by minimizing the number of nodes visited.

5.1. Number of Nodes Visited

Figure 2 shows a comparison between our proposed learned node selection policy and several others. From left to right, the first set of results are from using the expert policy defined in Section 4.3. The next set of results are from evaluating the learned policy discussed in Section 4.2, where the feature set included both the problem specific features and the branch-and-bound features. In the middle we include results from a similar learned policy, except that it only includes branch-and-bound features. The fourth set of results are from using the simple heuristic *best-first* search, which was discussed in Section 2.3. The final results, on the right hand side, are from a random selection policy.

From this comparison we see that our learned policy (second from the left) tends to outperform the other policies for the robotic motion planning problem. Since the learned policy was designed to imitate the expert policy, we would expect its performance to be lower bounded by the expert. We also see from these results that the simple *best-first* search also performs quite well. However it is not surprising that both learned policies outperform *best-first* because the node's lower bound value J is included as a feature input for the neural network, leading to the best-first search policy being contained within the class of learnable policies. Also, as expected the random policy performs vary poorly because it cannot leverage any problem structure.

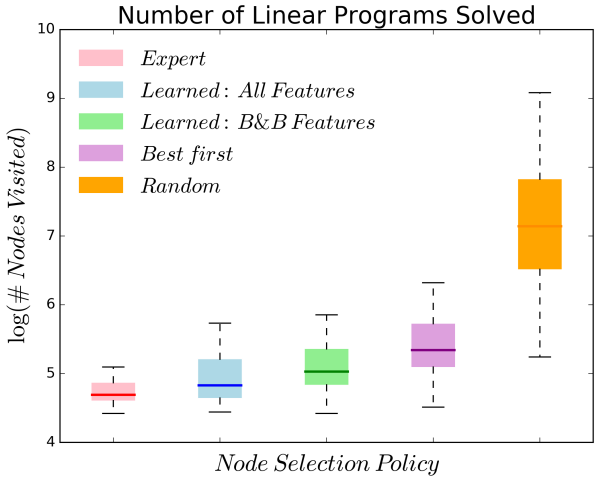

Figure 2. Comparison of several node selection policies by examining the number of nodes visited in the branch-and-bound algorithm. These are statistical results from over 800 robotic motion planning problem instances.

It is also important to note that it is possible for some problem instances for the learned policy to outperform the expert policy. This can occur because the solution to the MILP (1) is not necessarily unique, which means that while the expert guides the algorithm to explore a branch containing *one* global optima, it is possible that the learned policy "accidentally" finds a global optima on a different branch that leads to fewer nodes being visited. However in practice this does not happen too often, and the likelihood would probably decrease in more complex problems.

5.2. Algorithm Run Time

In addition to number of linear programs solved, we also look at the total run time of the branch-and-bound algorithm. The results of the comparison between the node selection policies is shown in Figure 3. The order of the policies is the same as in Figure 2. From left to right, the first results are from the expert policy. The second set of results are from the learned policy using both problem features and branch-and-bound features. In the middle is the learned

policy using only branch-and-bound features. The fourth set of results are from using the simple heuristic *best-first* search. Finally on the right are the results from the random selection policy.

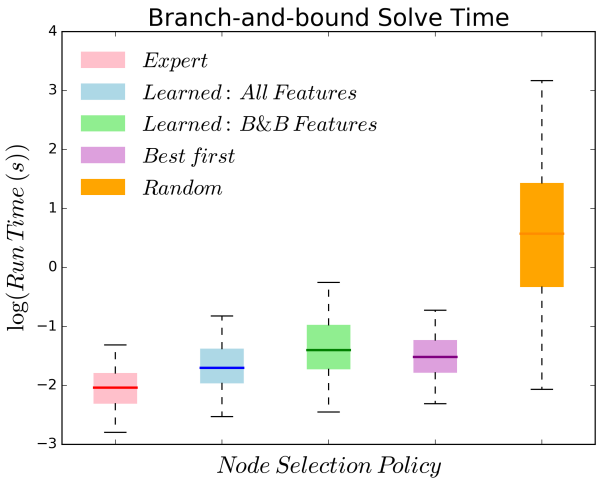

Figure 3. Comparison of several node selection policies by examining the total amount of time required to run the branch-and-bound algorithm. These are statistical results from over 800 robotic motion planning problem instances.

The trends here are similar to the results from the comparison of the number of linear programs solved, except that the advantage of the learned policy over the *best-first* policy has decreased (and in the case of the learned policy with only branch-and-bound features it appears *best-first* is outperforming). This highlights the critical trade-off between the performance gained by minimizing the number of nodes visited and the computational effort required to evaluate the policy at each node. *Best-first* is essentially the fastest possible policy to evaluate since the lower bound is already known, while the learned policies score a node by evaluating a multi-layer neural network. This could potentially limit the use of more complex function architectures for the learned policy, but the trade-off would have to be analyzed for each new problem (beyond the simple motion planning problem discussed here).

Note that in this work one technique that we used was to restrict our features to rely only on information local to a given node (or global information that remains fixed at each iteration, such as robot initial position). This allows us to avoid re-evaluating the network for *every* active node at each iteration of the algorithm, because a node's score will remain constant.

6. Conclusion

In conclusion, we have proposed a framework for using inverse RL to learn good node-selection heuristics for branch-and-bound algorithms solving a specific class of problems. We trained a neural-network function approximator to estimate the value of various nodes in the branch-and-bound tree in order to mimic the performance of an expert policy which already used the optimal solution. We further demonstrated our method on a class of robotic navigation problems, showing an improvement in both number of LPs solved, and solution time, over a commonly used (non-learned) heuristic strategy.

6.1. Future Work

This work presents a number of avenues for future research. The branch-and-bound algorithm contains two important decisions at each iteration: which node (partial solution) to branch, and which variable to branch on. This work studied how to learn good policies for node selection, but learning good variable selection policies remains an open question. Combining both of these approaches could yield much greater speedups than those from node selection alone.

Further, we hope to verify our problem on harder problem instances, including those with more time-steps, more obstacles, and harder classes of problems (including mixed-integer convex programs and mixed-integer nonlinear programs). Prior work has shown that learning can provide greater speedups in harder problem classes where simple heuristics often perform badly. This could hopefully lead to real-time hardware implementation, if our strategy can scale successfully.

Finally, it would be interesting to study policy transfer from small problems to large problems, to hopefully train a node selection policy on small problems (with few integer variables) and have it perform well on large problems of the same class. Such a transfer could be achieved through methods which allow various-sized inputs (such as convolutional architectures), and could allow for the generation of a large number of training examples, even when the large problem is hard to solve or intractable.

References

Alvarez, A. M., Louveaux, Q., and Wehenkel, L. A machine learning-based approximation of strong branching. *INFORMS Journal on Computing*, 29(1):185–195, 2017.

Amini, M. H., Frye, J., Ili, M. D., and Karabasoglu, O. Smart residential energy scheduling utilizing two stage mixed integer linear programming. In *2015 North American Power Symposium (NAPS)*, pp. 1–6, Oct 2015.

Balcan, M.-F., Dick, T., Sandholm, T., and Vitercik, E.

Learning to branch. *arXiv preprint arXiv:1803.10150*, 2018.

Bengio, Y., Lodi, A., and Prouvost, A. Machine learning for combinatorial optimization: a methodological tour d'horizon, 2018.

Conforti, M., Cornuejols, G., and Zambelli, G. *Integer Programming*. Springer Publishing Company, Incorporated, 2014.

Floudas, C. A. and Lin, X. Mixed integer linear programming in process scheduling: Modeling, algorithms, and applications. *Annals of Operations Research*, 139(1): 131–162, Oct 2005.

He, H., Daume III, H., and Eisner, J. M. Learning to search in branch and bound algorithms. In Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N. D., and Weinberger, K. Q. (eds.), *Advances in Neural Information Processing Systems 27*, pp. 3293–3301. Curran Associates, Inc., 2014.

Iglesias, R., Rossi, F., Wang, K., Hallac, D., Leskovec, J., and Pavone, M. Data-driven model predictive control of autonomous mobility-on-demand systems. In *Proc. IEEE Conf. on Robotics and Automation*, 2018.

Khalil, E. B., Bodic, P. L., Song, L., Nemhauser, G., and Dilkina, B. Learning to branch in mixed integer programming. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, AAAI'16, pp. 724–731. AAAI Press, 2016.

Landry, B., Deits, R., Florence, P. R., and Tedrake, R. Aggressive quadrotor flight through cluttered environments using mixed integer programming. In *Robotics and Automation (ICRA), 2016 IEEE International Conference on*, pp. 1469–1475. IEEE, 2016.

Lodi, A. and Zarpellon, G. On learning and branching: a survey. *TOP*, 25(2):207–236, 2017.

Mashayekh, S., Stadler, M., Cardoso, G., and Heleno, M. A mixed integer linear programming approach for optimal der portfolio, sizing, and placement in multi-energy microgrids. *Applied Energy*, 187:154 – 168, 2017.

Pinto, J. M. and Grossmann, I. E. A continuous time mixed integer linear programming model for short term scheduling of multistage batch plants. *Industrial & Engineering Chemistry Research*, 34(9):3037–3051, 1995.

Pochet, Y. and Wolsey, L. *Production Planning by Mixed Integer Programming*. Springer Series in Operations Research and Financial Engineering. Springer New York, 2006.

Richards, A., Schouwenaars, T., How, J. P., and Feron, E. Spacecraft trajectory planning with avoidance constraints using mixed-integer linear programming. *Journal of Guidance, Control, and Dynamics*, 25(4):755–764, 2002.

Ross, S., Gordon, G., and Bagnell, D. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*, pp. 627–635, 2011.

Sabharwal, A., Samulowitz, H., and Reddy, C. Guiding combinatorial optimization with uct. In Beldiceanu, N., Jussien, N., and Pinson, É. (eds.), *Integration of AI and OR Techniques in Contraint Programming for Combinatorial Optimzation Problems*, pp. 356–361, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

Schouwenaars, T., De Moor, B., Feron, E., and How, J. Mixed integer programming for multi-vehicle path planning. In *European Control Conference*, 2001.

Tsao, M., Milojevic, D., Ruch, C., Salazar, M., Frazzoli, E., and Pavone, M. Model predictive control of ride-sharing autonomous mobility on demand systems. In *Proc. IEEE Conf. on Robotics and Automation*, 2019. In Press.